

# Algoritmul fazelor

```
chan ch[1:n](int id, tok_type tok);
const int D = diametrul_retelei;

process Proc[p = 1 to n] {
  bool in[1:n] = in-vecinii_lui_p;
  bool out[1:n] = out-vecinii_lui_p;
  int rec[1:n] = ([n] 0);
  /* rec[q] = număr mesaje primite de la q */
  int sent = 0;
  /* număr de mesaje transmise fiecărui out-vecin */
  tok_type tok;
  int id;
```



# Algoritmul fazelor (2)

```
  if (p este inițiator) {
      for [q = 1 to n st out[q]] send ch[q](p, tok);
      sent = sent + 1
  }

  /* min(rec) este min(rec[q], ∀q ∈ in) */
  while (min(rec) < D) {
      receive ch[p](id, tok); rec[id] = rec[id] + 1;
      if (min(rec) >= sent and sent < D) {
            for [q=1 to n st out[q]] send ch[q](p, tok);
            sent = sent + 1;
      }
  }
  decide;
}
```

**Mesaje = 2D|E|, unde E este mulț. canalelor nedirijate (2 canale dirijate)**
**Timp = 2D**

# Algoritmul fazelor pentru clici

```
chan ch[1:n](int id, tok_type tok);
process Proc[p = 1 to N] {
   bool Vecini[1:n] = vecinii_lui_p;
   int n_vecini = numar-vecini_p;
   int rec = 0, sent = 0, id;
   tok_type tok;
   if (p este initiator) {
        for [q=1 to N st Vecini[q]] send ch[q](p, tok);
        sent = sent+1;
   }
   while (rec < n_vecini) {
        receive ch[p](id, tok); rec = rec + 1;
        if (sent == 0) {
               for [q=1 to N st Vecini[q]] send ch[q](p, tok);
               sent = sent + 1;
   }    }
   decide;
}
```

$$\boldsymbol{Mesaje = N(N-1), Timp = 2}$$